ELK Movies Data Platform

Document Synthèse

Plateforme d'analyse de films avec la stack Elasticsearch · Logstash · Kibana

Mathurin Bernonville
Master 1 Data & IA — HETIC
Année universitaire 2025-2026

Projet individuel — Examen ELK

1. Contexte, objectifs et périmètre

1.1 Problème traité

Ce projet consiste à construire une plateforme complète d'analyse de données cinématographiques en exploitant la stack ELK (Elasticsearch, Logstash, Kibana). L'enjeu principal est de transformer un dataset CSV brut de plus de 660 000 films en une base de données exploitable, interrogeable en temps réel et visualisable via un dashboard métier.

Le dataset utilisé est le dataset de référence imposé par le sujet : movies.csv issu de Kaggle (Akshay Pawar, "Millions of Movies", version 67), contenant des informations sur 662 083 films internationaux issus de la base TMDB.

1.2 Objectifs techniques et métier

- Mettre en place une stack ELK reproductible via Docker Compose
- Ingérer les données brutes dans un index movies_raw sans transformation
- Nettoyer, typer et normaliser les données dans un index movies_clean avec mapping explicite
- Produire 12 requêtes Elasticsearch DSL commentées dont 5 requêtes bool
- Créer un dashboard Kibana avec 8 visualisations répondant à des questions métier
- Développer un moteur de recherche Flask connecté à Elasticsearch
- Documenter l'ensemble du projet de manière reproductible

1.3 Périmètre réalisé vs non réalisé

Fonctionnalité	Statut	Commentaire
Stack ELK Docker Compose	✅ Réalisé	Elasticsearch + Kibana + Logstash 8.13.0
Ingestion movies_raw	✅ Réalisé	662 083 documents indexés
Nettoyage movies_clean	✅ Réalisé	662 083 documents nettoyés
Mapping explicite + analyzer	✅ Réalisé	movie_text_analyzer personnalisé
12 requêtes DSL (dont 5 bool)	✅ Réalisé	Fichier queries/elasticsearch_queries.md
Dashboard Kibana (8 visu)	✅ Réalisé	Export .ndjson disponible
Moteur de recherche Flask	✅ Réalisé	Interface UI avec filtres et tri
Posters des films	❌ Non réalisé	Champ poster_path exclu du pipeline clean

2. Architecture et environnement

2.1 Stack technique

Service	Image Docker	Port	Rôle
Elasticsearch	docker.elastic.co/elasticsearch:8.13.0	9200	Moteur d'indexation et de recherche
Kibana	docker.elastic.co/kibana:8.13.0	5601	Visualisation et dashboard
Logstash	docker.elastic.co/logstash:8.13.0	—	Ingestion et transformation des données
Flask (Python)	python:3.12 (local)	5000	Moteur de recherche UI

2.2 Flux de données

Le flux de données suit une architecture linéaire en quatre étapes :

- movies.csv (662 083 lignes) est monté en lecture seule dans le conteneur Logstash
- Logstash parse le CSV via deux pipelines distincts : 01_movies_raw.conf (ingestion brute) et 02_movies_clean.conf (nettoyage et normalisation)
- Elasticsearch stocke les données dans deux index : movies_raw (données brutes) et movies_clean (données typées et normalisées)
- Kibana consomme movies_clean pour produire visualisations et dashboard
- Le moteur Flask interroge movies_clean via l'API Elasticsearch REST

2.3 Volumes et configuration Docker

- ./logstash/pipeline → /usr/share/logstash/pipeline:ro (pipelines en lecture seule)
- ./DATA → /data:ro (dataset CSV en lecture seule)
- ./logs → /usr/share/logstash/logs (logs Logstash)
- esdata (volume nommé Docker) → persistance des données Elasticsearch

La sécurité xpack est désactivée (xpack.security.enabled=false) pour simplifier l'environnement de développement local.

3. Données et nettoyage (obligatoire)

3.1 Description des colonnes du dataset

Champ	Type source	Type ES (clean)	Description
id	string	integer	Identifiant unique TMDB
title	string	text (analyzed)	Titre du film
genres	string (tirets)	keyword[]	Genres séparés par des tirets
original_language	string	keyword	Code ISO de la langue originale
overview	string	text (analyzed)	Synopsis du film
popularity	string	float	Score de popularité TMDB
production_companies	string (tirets)	keyword[]	Sociétés de production
release_date	string	date	Date de sortie (yyyy-MM-dd)
budget	string	float	Budget en USD (0 = inconnu)
revenue	string	float	Revenus en USD (0 = inconnu)
runtime	string	float	Durée en minutes
status	string	keyword	Statut de sortie
tagline	string	text (analyzed)	Slogan du film
vote_average	string	float	Note moyenne (0-10)
vote_count	string	float	Nombre de votes
credits	string (tirets)	keyword[] (cast)	Acteurs extraits (top 5)
keywords	string (tirets)	keyword[]	Mots-clés associés

3.2 Anomalies détectées

L'analyse du dataset a révélé plusieurs catégories d'anomalies :

- Valeurs numériques à 0 pour budget, revenue et runtime : représentent des données manquantes dans le dataset TMDB (~60% des films). Ces valeurs sont conservées car elles sont techniquement valides dans le CSV.
- Valeurs textuelles "nil" dans certains champs numériques : quelques lignes contiennent la chaîne de caractères "nil" au lieu d'une valeur numérique, causant des erreurs de parsing Elasticsearch. Traitées en amont dans le pipeline Ruby.
- Champs liste encodés en chaîne de caractères : les champs genres, keywords, production_companies et credits utilisent le tiret (-) comme séparateur au lieu du format JSON standard. Normalisés en tableaux par le filtre Ruby.
- Dates manquantes ou nulles : environ 5% des films n'ont pas de date de sortie. Le filtre date de Logstash les marque avec le tag _date_parse_failure sans bloquer l'ingestion.
- Encodage UTF-8 avec caractères spéciaux : les noms d'acteurs et titres contiennent des caractères accentués parfois mal encodés (ex : Ã¥ au lieu de å). Conservés tels quels car la correction nécessiterait une intervention manuelle.

3.3 Règles de nettoyage appliquées dans Logstash

Le pipeline 02_movies_clean.conf applique les transformations suivantes dans cet ordre :

- Étape 1 — Suppression des lignes invalides : tout document sans id ou sans title est supprimé via drop {}
- Étape 2 — Nettoyage préventif Ruby : les valeurs nil textuelles et les chaînes vides sont supprimées des champs numériques avant toute conversion de type
- Étape 3 — Conversion de types : id (integer), popularity/vote_average (float), budget/revenue/runtime/vote_count (float)
- Étape 4 — Parse de date : release_date converti en type date Elasticsearch avec le pattern yyyy-MM-dd
- Étape 5 — Normalisation des listes : genres, keywords, production_companies et credits sont splitté sur le séparateur tiret et convertis en tableaux de strings
- Étape 6 — Extraction du cast : les 5 premiers acteurs sont extraits du champ credits et stockés dans un nouveau champ cast
- Étape 7 — Suppression des champs inutiles : poster_path, backdrop_path, recommendations et credits sont supprimés de movies_clean

3.4 Mesure d'impact avant/après nettoyage

Métrique	movies_raw	movies_clean
Nombre de documents	662 083	662 083
Type du champ genres	string CSV (tirets)	keyword[] (tableau)
Type du champ release_date	string	date ES (triable)
Type du champ vote_average	string	float (agrégeable)
Valeurs nil textuelles	Présentes → erreurs ES	Supprimées avant conversion
Champ cast dédié	Absent	keyword[] (top 5 acteurs)
Recherche full-text optimisée	Non (pas d'analyzer)	Oui (movie_text_analyzer)
Agrégations possibles	Limitées (types string)	Complètes (types natifs)

4. Modélisation Elasticsearch

4.1 Analyzer personnalisé : movie_text_analyzer

Un analyzer personnalisé movie_text_analyzer a été défini dans les settings de l'index movies_clean. Il est composé des éléments suivants :

- Tokenizer standard : découpe le texte sur les espaces et la ponctuation
- Filter lowercase : normalise tout en minuscules
- Filter asciifolding : convertit les caractères accentués en ASCII (é → e, ü → u)
- Filter movie_stop_words : supprime les mots vides anglais courants (the, a, an, and, or, of, in, to)
- Filter movie_stemmer : applique un stemming en anglais (running → run, movies → movi)

Justification : l'asciifolding permet de retrouver un film en tapant "Amelie" au lieu de "Amélie". Le stemming améliore le recall sur des requêtes comme "space" qui retrouve aussi "spaces" ou "spacing".

Un second analyzer keyword_lowercase est défini pour les recherches exactes insensibles à la casse sur des champs comme le titre.

4.2 Choix text vs keyword

Champ	Type	Justification
title	text + keyword (multi-field)	Recherche full-text ET tri/agrégation exact
overview, tagline	text	Recherche full-text uniquement
genres, keywords, cast	keyword	Filtres exacts et agrégations (terms)
original_language, status	keyword	Valeurs discrètes, filtres exacts
release_date	date	Tri chronologique et range queries
vote_average, popularity	float	Calculs statistiques et tri
budget, revenue	long	Valeurs potentiellement très grandes

5. Requêtes et analyses

5.1 Requêtes principales

12 requêtes DSL ont été développées et commentées dans le fichier queries/elasticsearch_queries.md. Les 5 requêtes bool illustrent différentes combinaisons de clauses :

ID	Type	Cas métier
R01	Count	Vérification post-ingestion : 662 083 documents
R02	Match	Recherche full-text avec analyzer personnalisé
R03	Range + Sort	Top films les mieux notés (vote_count ≥ 500)
R04	Bool (must + filter)	Films d'action en anglais sortis après 2010
R05	Bool (must + must_not)	Films populaires mais peu votés (niche)
R06	Bool (multi_match + filter)	Recherche full-text avec filtres langue et note
R07	Bool (must + filter + must_not)	Flops financiers : budget > 100M, revenue < 50M
R08	Bool (should)	Comédies ou romances bien notées (≥ 7.5)
R09	Aggregation terms	Nombre de films par genre (Top 20)
R10	Aggregation imbriquée	Note moyenne et nb films par langue
R11	Date histogram	Distribution des sorties par année
R12	Script sort (Painless)	Top films par ROI (revenue / budget)

5.2 Résultats notables

- R04 (films d'action en anglais post-2010) : Avengers: Endgame en tête avec une popularité de 283, suivi de Spider-Man: No Way Home
- R07 (flops financiers) : plusieurs blockbusters avec budget > 200M USD et revenues inférieurs à 50M USD, dont des productions récentes
- R12 (ROI) : des films à très petit budget (< 1M USD) affichant des ROI extraordinaires, typiques du cinéma indépendant
- R11 (sorties par année) : pic de production entre 2010 et 2020, avec un creux visible en 2020-2021 correspondant à la période COVID

6. Dashboard Kibana et lecture métier

6.1 Visualisations produites

ID	Titre	Type	Insight métier
V1	Films par genre	Bar chart	Drama et Comedy dominent le catalogue mondial
V2	Distribution des notes	Histogram	Concentration autour de 6-7/10, courbe en cloche
V3	Sorties par année	Area chart	Croissance exponentielle depuis 1980, pic 2015-2019
V4	Top langues	Pie chart	L'anglais représente ~60% des films indexés
V5	Note moyenne par genre	Bar horizontal	Documentary et History ont les meilleures notes moyennes
V6	Films par statut	Donut	95%+ des films ont le statut Released
V7	Note moyenne par langue	Bar chart	Les films en langues moins répandues ont souvent de meilleures notes
V8	Top films populaires	Table	Avengers Endgame et Spider-Man en tête de popularité TMDB

6.2 Limites d'interprétation

- Les notes à 0 (films sans votes) ont été exclues des visualisations V2, V5 et V7 via un filtre Kibana pour éviter de biaiser les moyennes
- La popularité TMDB est un score propriétaire qui favorise les films récents et les grandes franchises hollywoodiennes
- Les budgets et revenues à 0 représentent des données manquantes et non des films sans budget, ce qui limite l'analyse financière
- Le dataset étant issu de TMDB, il surreprésente les productions anglophones et américaines

7. Gestion de projet et collaboration

7.1 Organisation Gitflow

Le projet a été réalisé individuellement. Le Gitflow a été maintenu pour démontrer une démarche professionnelle et assurer la traçabilité du travail. Les branches suivantes ont été créées et mergées via Pull Request :

Branche	Contenu	PR vers
main	Version stable finale	—
dev	Branche d'intégration	main
feature/bootstrap-stack	Docker Compose, structure du projet	dev
feature/cleaning-mapping	Pipeline nettoyage + mapping ES	dev
feature/queries	12 requêtes DSL commentées	dev
feature/kibana-dashboard	Export .ndjson du dashboard	dev
feature/search-engine	Moteur de recherche Flask	dev
feature/documentation	Tous les fichiers docs/*.md	dev

7.2 Planning Poker

En contexte solo, le planning poker a été simulé en adoptant plusieurs perspectives (développeur, data engineer, utilisateur final) pour estimer l'effort de chaque user story sur l'échelle de Fibonacci (1, 2, 3, 5, 8, 13).

Story	Estimation	Réalité
US-01 Bootstrap Docker	2 pts	Conforme — Docker déjà maîtrisé
US-02 Ingestion brute	3 pts	Conforme — CSV standard
US-03 Nettoyage clean	5 pts	Sous-estimé — format CSV non-JSON inattendu
US-04 Mapping + analyzer	3 pts	Conforme
US-05 Requêtes DSL	3 pts	Conforme
US-06 Dashboard Kibana	5 pts	Conforme
US-07 Moteur de recherche	5 pts	Conforme — Flask maîtrisé
US-08 Documentation	3 pts	Conforme

Total estimé : 29 points. La story US-03 a été la plus complexe en raison du format CSV inattendu (champs séparés par des tirets au lieu de JSON), nécessitant une réécriture complète du filtre Ruby.

8. Bilan, limites et améliorations

8.1 Retour d'expérience

Ce projet a permis de maîtriser l'ensemble de la stack ELK dans un contexte réaliste : ingestion d'un large dataset, nettoyage avec Logstash, modélisation avec des mappings Elasticsearch avancés, et restitution via Kibana et une interface web custom.

La principale difficulté rencontrée a été la découverte du format réel du CSV (champs encodés en tirets) qui ne correspondait pas au format JSON attendu, nécessitant une adaptation complète du pipeline de nettoyage. Cette situation illustre l'importance de l'étape d'exploration des données avant de concevoir le pipeline ETL.

8.2 Points bloquants rencontrés

- Valeurs "nil" textuelles dans les champs numériques : causaient des rejets ES (status 400). Résolu par un filtre Ruby préventif supprimant ces valeurs avant la conversion de type.
- Format CSV non-JSON pour les champs liste : genres, credits, keywords encodés avec des tirets au lieu de JSON. Résolu par un split Ruby sur le séparateur tiret.
- Conflit de version Docker Compose : l'attribut version est obsolète en Compose v2, générant des warnings. Sans impact sur le fonctionnement.

8.3 Axes d'amélioration

- Intégrer les posters des films via l'API TMDB pour enrichir l'interface de recherche
- Ajouter une logique de scoring personnalisée dans le moteur de recherche (boost sur vote_count, popularité)
- Implémenter des suggestions (autocomplete) via un index de completion Elasticsearch
- Comparer plusieurs analyzers (standard, french, english) pour mesurer l'impact sur la précision des recherches
- Mettre en place un pipeline d'ingestion incrémentale pour ne réingérer que les nouveaux films
- Ajouter des tests automatisés sur les requêtes DSL pour valider les résultats attendus

Repo GitHub : https://github.com/MathurinBernonville/elk-movies